CHC THESIS COVER SHEET

Name: Shreya Nakum

Thesis Title: Using Convolutional Neural Networks (CNNs) to Estimate Female Fecundity in Populations of *Drosophila Melanogaster*

Faculty Advisor + Department: Michael R. Rose, Department of Ecology & Evolutionary Biology and Larry D. Mueller, Department of Ecology & Evolutionary Biology

Graduating Quarter + Year: Spring 2027

Using Convolutional Neural Networks (CNNs) to Estimate Female Fecundity in Populations of *Drosophila Melanogaster*

Nakum, Shreya V.

Department of Ecology & Evolutionary Biology

Michael R. Rose, PhD & Larry D. Mueller, PhD

Abstract

Measuring female fecundity, the number of eggs laid by *Drosophila melanogaster*, is a standard method for estimating biological fitness, but manual counting at scale is very time-consuming. We present a convolutional neural network (CNN) approach that automates this process using a recursive tiling strategy, where caps are decomposed into 75 by 75 pixel tiles that are individually classified and summed to produce a total egg count per cap. This decomposition reduces the classification space from hundreds of cap-level classes to 43 tile-level classes, making training faster. We trained and evaluated multiple CNNs across two independent fecundity datasets, a Lithium assay dataset and an external desiccation-resistance dataset. The best-performing model on the Lithium test set achieved an root mean squared error (RMSE) of 7.55 eggs per cap ($R^2 = 0.91$), while the best model on the external dataset achieved an RMSE of 14.29 eggs per cap ($R^2 = 0.63$). This model preserved the population-level fecundity difference between experimental groups in the external dataset despite never having seen those styles of images during training. We further developed a Python-based command line tool that automates the full pipeline from cap images to a comma-separated values (CSV) file of egg counts per cap, reducing processing time from thousands of hours of manual labor to days of computation. These results show that CNN-based tiling is a viable and scalable approach for large-scale fecundity assays in evolutionary biology research.

1 Introduction

In the field of computational biology, there has been an increasing interest in finding methods to quantify and measure personal health and well-being. One method to measure this is biological fitness, which refers to the overall survival and reproductive success of an organism (Wadgymar et al., 2024). In particular, fitness can be estimated by measuring two life-history characteristics: fecundity (the amount of offspring produced by individuals in a population) and mortality (the number of individuals who die at a given time point in a population).

In the context of *Drosophila melanogaster*, female fecundity is defined as the number of eggs laid by a given female, or the females in a population. Female fecundity is a common assay used to estimate fitness. For instance, it has been used in experiments measuring the decline in fitness from early to late life stages (Rauser et al., 2003), relative fitness of populations at different ages under different environmental conditions (e.g. diet) (Rutledge et al., 2020), and the trade-offs between early fecundity and longevity (Djawdan et al., 1996). All of these studies have profound implications for human health, and were built in part on the ability to measure fitness through the proxies of fecundity and mortality. However, current methods to accomplish this can be extremely slow and time consuming.

Previously, there have been a variety of techniques to measure fecundity, one of which is by keeping flies in food-containing vials, on which eggs, which are white and will sharply contrast with the black background, can easily be identified and manually counted by humans (Mueller and Rose, 2007). Historically, researchers have relied on using programs like ImageJ to manually click and tally each egg in a grid of images. While this works for small sample sizes, fecundity assays with larger sample sizes can take a prohibitively long time to count with a limited workforce–on the order of hundreds to thousands of hours spent on counting alone. This is a major limitation for anyone aiming to do large-scale fecundity experiments. For instance, if a project is comparing 20 populations, measuring 50 flies per population per day, over a course of 20 days, you end up with 20,000 images. That means that even at 3 minutes of counting per image, it would take 1,000 hours to complete that project. This would be a large diversion of resources from standard lab work like maintaining the populations, and may also introduce some error if less trained research assistants differ in what they qualify as an egg, and thus give different counts. Since this lab has a significant backlog of data from experiments of that scale, and intends to do more in

the future, there is a strong impetus to find better methods for processing fecundity data.

Ultimately, our objective was to develop an automatic way of counting these fecundity caps. In doing this, the data can quickly be processed and students can instead focus on the more important and interesting work of research. We found that developing a convolutional neural network (CNN) with a recursive "tiling" process of detection allowed for this problem to be addressed. Our approach consisted of first creating multiple training sets with tiles–small, square pieces of the to-be-counted images–counted by different people. We then trained multiple CNNs based on various combinations of the training sets. We then tested the model on two sets, on which was counted via tiles and the second, larger set which was counted via caps and used in Phillips et al., 2018. The model that performed best on the first set was, on average, off by 7.55 eggs on the caps and the model that performed best on the second set was, on average, off by 14.29 eggs on the caps. Following these results, we developed a command line tool to allows for usage of our models to quickly get the counts of fecundity data.

2 Definitions

In this work, we define **cap** to be the shallow circular dish containing *Drosophila* eggs. We define **tile** to be a 75 by 75 pixel piece of an image of a cap. We define **grid image** to be the original complete image consisting of 5 rows and 4 columns of caps. We refer to the convolutational neural network developed as the **CNN**. We define **cluster** as a tile containing a highly dense amount eggs, typically 10 or more eggs. We refer to the **Lithium dataset** as the fecundity data gathered from the Lithium assays. We refer to the **CD dataset** as the fecundity data collected in Phillips et al., 2018. We define **over-counting** as the act, performed by either a model or human, of counting more eggs than the average person would in a clustered tile or high-egg-count cap. We define the **training set** as the data used to train the CNN and the **test set** as the data held out to evaluate model performance after training. We refer to **Cohen's Kappa** (κ) as a statistical measure of inter-rater agreement between two counters, where $\kappa = 1$ indicates perfect agreement and $\kappa = 0$ indicates agreement no better than chance. We refer to the **command line tool** as the Python-based CLI developed to automate the transition from raw grid images to a CSV file of egg counts. We have examples of grid, cap, and tile images in Appendix A.

3 Related Work

Biology is a data-heavy field, where a substantial amount of data processing is images. As seen by Lürig et al., 2021, the field has begun integration of computer vision (CV) and machine learning, which allows researchers to transcend the "phenotypic bottleneck," the gap between the high-speed sequencing of genomes and the labor intensive manual recording of biological traits. Our work follows this trajectory by applying deep learning to automate the quantification of fitness-related traits in *Drosophila*.

3.1 Classical and Density-Based Counting

In the past, automatic fecundity identification was done through classical imaging techniques. A significant milestone in this field was the development of *QuantiFly*, introduced by Waithe et al., 2015. This software used a density-estimation approach based on supervised learning to count eggs on varying substrates. While highly effective, *QuantiFly* often required specific imaging conditions (e.g., specific lighting or agar contrast) and sometimes struggled with high-density egg clusters or complex visual noise.

Density-based counting has a longer history in computer vision. Methods such as multi-column convolutional neural networks (MCNN) (Zhang et al., 2016) and CSRNet (Li et al., 2018) were originally developed for crowd counting and later applied to biological object counting. These architectures generate a density map over the image, and the integral of that map gives the count. While this works well in some cases, these methods require dense per-pixel annotations to produce ground-truth density maps, which is not practical for tile-level egg counting where the label is a total count per tile.

3.2 Segmentation-Based Approaches

Beyond density estimation, the field shifted toward semantic and instance segmentation. The U-Net architecture by Ronneberger et al., 2015 proved useful for biomedical imaging because it can work with little training data and still be accurate. In *Drosophila* research, variants of U-Net have been used to distinguish biological debris from actual eggs. However, these pixel-level approaches still struggle when eggs overlap in dense regions.

3.3 End-to-End Detection Frameworks

Recent work has moved toward end-to-end deep learning frameworks for population-level assays. Gabidulin and Rudman, 2023 introduced Machine Learning Data Acquisition for Assessing Population Phenotypes (MLDAAPP), which uses the YOLOv8 architecture for real-time object detection, achieving around 91% accuracy in identifying eggs and other phenotypic markers. Unlike sliding-window approaches, YOLO treats object detection as a single regression problem, going directly from pixels to bounding box coordinates and class probabilities.

3.4 Transfer Learning in Biological Imaging

Transfer learning has been shown to reduce the amount of labeled biological data needed to reach a given accuracy (Lürig et al., 2021). In our work, we trained CNN weights from scratch rather than fine-tuning a pre-trained model. We did this because our input tiles are 75×75 pixels, which is very different from the high-resolution photos that ImageNet models are trained on.

Despite the success of models like YOLO, dense egg clusters remain a challenge. Detection-based methods like MLDAAP provide spatial localization, but regression-based methods tend to do better when eggs are densely packed.

4 Methods

In this section, we go over the model's training and its implementation. Our user guide guide that includes information on command line tool usage and model implementation is available on our GitHub.

4.1 Input Preparation (Tiling)

For the CNN to predict egg counts, the input images provided must align with the tiling process, the recursive method through which eggs are counted.

The tiling process refers to the decomposition of the grid image down to tiles, as seen in Figure 1. Prior to model inference, the grid images are segmented into 800 by 800 pixel individual caps. Each cap contains spatial coordinates with a unit length of one referring to 75 pixels. These coordinates are

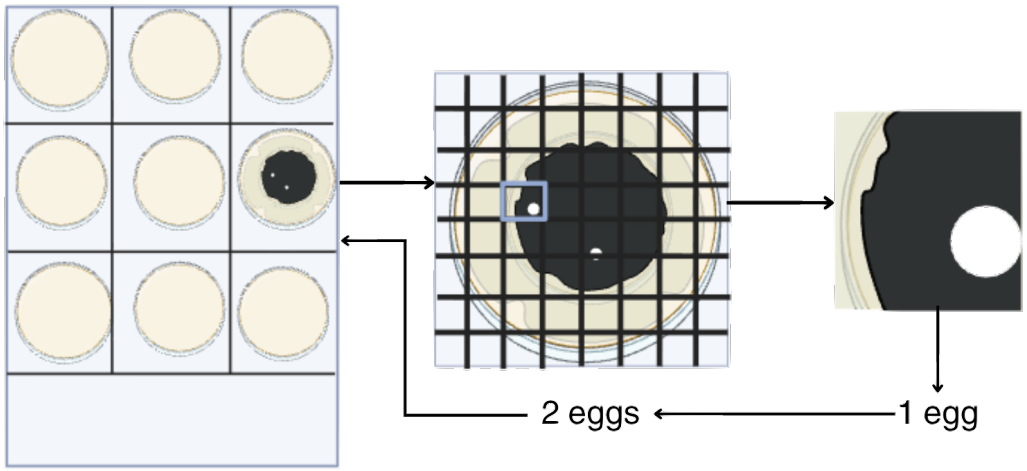


Figure 1: Tiling process

determined based on the origin being the upper left corner as defined by Buitinck et al., 2013. Next, each cap is subdivided into tiles. Tiles that are whitespace of the gridded background are automatically excluded, as they contain 0 eggs and have no impact on later prediction. We determine tiles with only whitespace based on their average file size, as whitespace tiles have a smaller file size than tiles with eggs.

This tiling approach solves the "many classes" issue, where caps can have egg counts ranging from 0 to 200, where even caps with the same number of eggs can look drastically different due to where eggs are located on the cap, leading to hundreds of classes. By breaking the image into tiles, we limit the classes and appearance of them to 43, where the majority of tiles lay between 0 and 10 eggs as seen in Figure 2.

4.2 Data

Our data consisted of grid images from two independent fecundity assays, the Lithium assay and the desiccation-resistant *Drosophila* data from Phillips et al., 2018, which were both conducted in the same laboratory setting. Egg counts were obtained both via manual tile counting and cap counting.

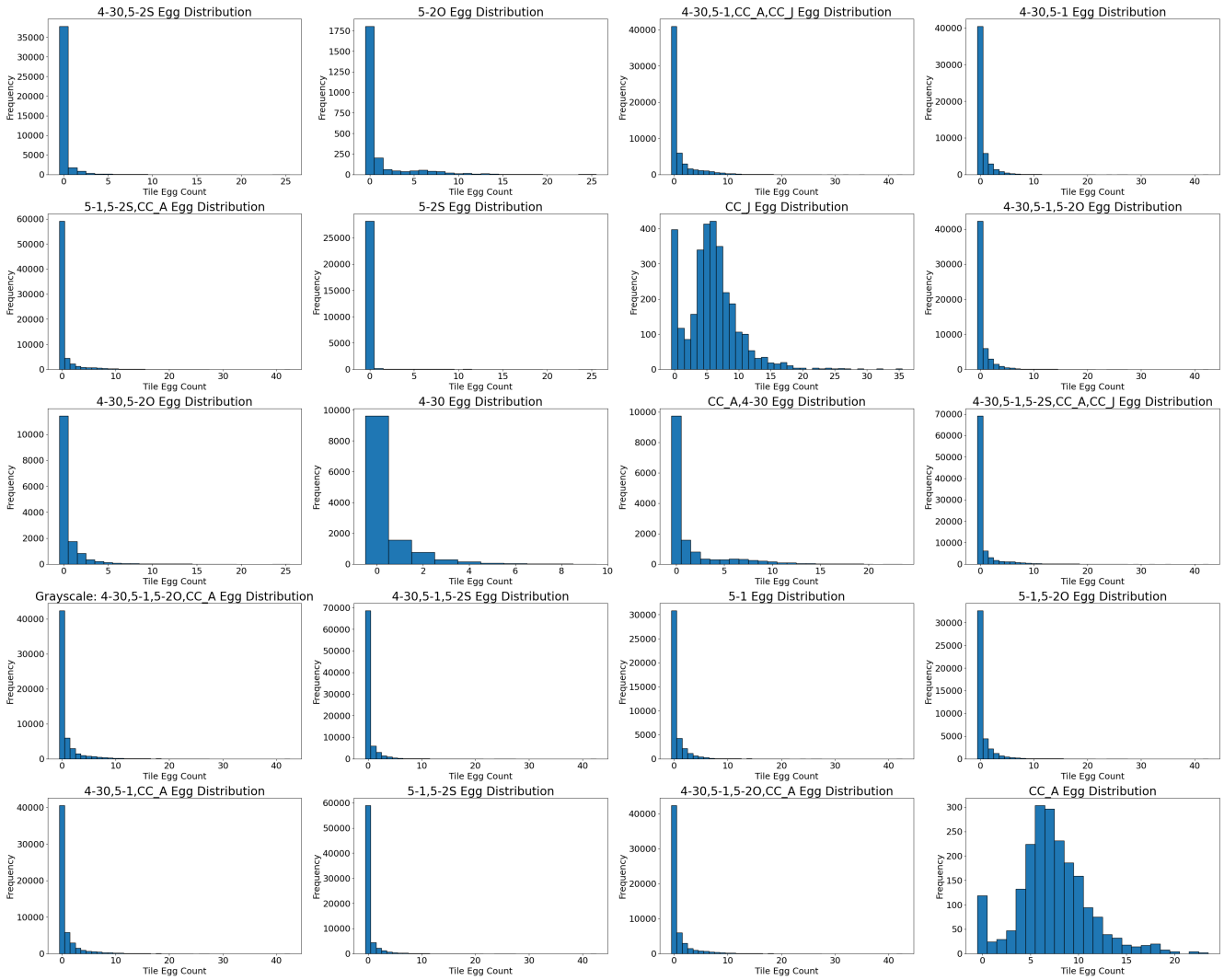


Figure 2: Distribution of Eggs in Training Sets

Through manual tile counting, we obtained four sets titled by month and day that the caps were collected for the Lithium fecundity assay: 4-30, 5-1, 5-2, and 5-4. Additionally, we developed two smaller cluster sets which only contain highly dense clustered tiles. Each of the sets is counted by manual counters as follows:

Where two counters annotated the same set, we computed Cohen's Kappa (κ) to quantify inter-rater agreement. For the 5-4 dataset, $\kappa = 0.7292$, which is conventionally interpreted as *substantial* agreement (Landis and Koch, 1977). This value contextualizes the inherent difficulty of the task: even trained human counters do not agree perfectly, particularly on cluster tiles where egg boundaries are ambiguous. As a consequence, our model is not optimized to match a single ground-truth count but rather to produce predictions within the range of counts a typical human would report or to follow the average human's

Dataset Name	Number of Tiles	Counter
4-30	12,510	Alexander
5-1	40,024	Alexander
5-2 O-less (5-2O)	2,449	Alexander and Marvin
5-2 Supplemented (5-2S)	28,780	Alexander and Marvin
5-4	40,001	Julie and Angela
Cluster Counts Jacob (CC_J)	3,141	Jacob
Cluster Counts Alexander (CC_A)	2,100	Alexander

Table 1: Datasets used to create training sets for the CNN. Note that the 5-2S set contains 5-2O plus more non-whitespace 0-count tiles.

counts. Thus, some datasets were counted by two people because the objective of this model is to have predictions between the two people–the model is meant to be an average human.

Figure 2 displays the distribution of egg counts across tiles in each training set. The majority of tiles contain between 0 and 5 eggs, with frequency dropping sharply as egg count increases. This heavy right skew reflects the biological reality of fecundity assays: most regions of a cap contain few or no eggs, while densely clustered regions are comparatively rare. The skew also highlights the class imbalance challenge inherent in training. Since zero and low-count tiles dominate the dataset, models trained without careful dataset curation risk developing a bias toward under-prediction on sparse caps while simultaneously struggling on the rare high-density tiles. The inclusion of supplemented zero-count tiles in the 5-2S dataset and dedicated cluster sets (CC_A, CC_J) were motivated by this imbalance, as these additions provide the model with additional exposure to the underrepresented tails of the distribution.

With these sets, 4-30, 5-1, and 5-2 were used for training while 5-4 was held out for testing. Alongside the manually counted tile set, we also obtained the **CD** dataset's cap counts used in Phillips et al., 2018 which contains 839,600 tiles (8,395 caps) to test the model.

4.3 Model Training

The training phase was executed using TensorFlow and Keras frameworks (Martín Abadi et al., 2015). We developed a series of CNNs to evaluate how different training sets influenced predictive accuracy.

4.3.1 Architecture

Figure 3 shows the CNN architecture used in this study. Each CNN takes 75×75 RGB tiles as input and passes them through three convolutional layers, each using ReLU activation to learn spatial features. After the first and second convolutions, max-pooling layers (pool size 2×2) reduce spatial dimensionality. We used max pooling instead of average pooling because egg detection relies on local high-contrast features, the white egg against the dark background, and max pooling preserves the strongest activation in each region. After the third convolutional layer, the feature maps are flattened and passed to a dense layer with 64 neurons, followed by a final output layer with 43 neurons (one per egg count from 0 to 42) using softmax activation. We chose three convolutional layers and 64 neurons by testing two and four layers and widths of 32, 64, and 128.

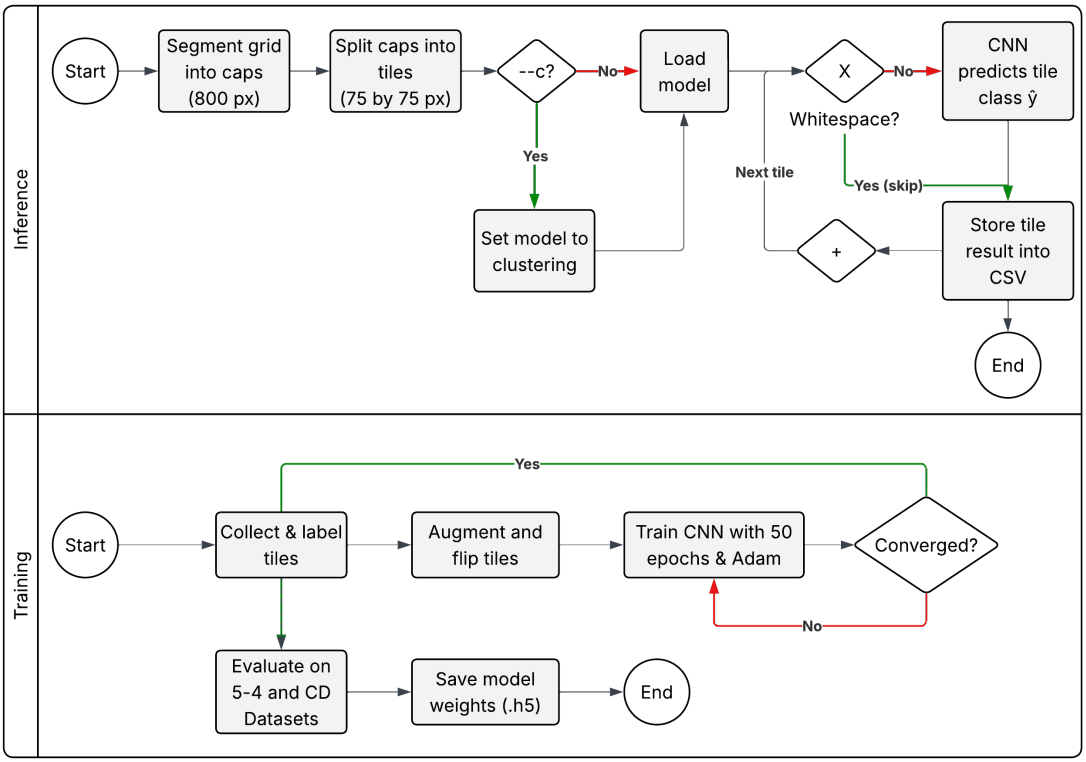


Figure 3: CNN architecture. Input tiles ($75 \times 75 \times 3$) pass through three convolutional layers interleaved with two max-pooling layers, then through a 64-neuron dense layer and a 43-class softmax output layer.

4.3.2 Training Procedure and Hyperparameters

Each model was trained for 50 epochs with a batch size of 32. We split the training data in an 80/20 stratified split to keep consistent class distributions. Moreover, we measured performance with Mean Square Error (MSE), Root Mean Square Error (RMSE), and R^2 scores. We use R^2 since a model that performs exactly as the human does should have a linear, 1:1 relationship with the correct counts. As stated earlier, this problem can be treated as a regression problem since the classes are numerical.

Tiles can come in many different variations of positioning. For instance, a tile with 3 eggs can have those 3 eggs positioned anywhere on it. To address this, we used the `ImageDataGenerator` class to augment tiles through rotations up to 20 degrees, horizontal and vertical shifts up to 10% of the original width, and horizontal and vertical flips to account for various egg orientations.

We used the Adam optimizer with its default learning rate ($\eta = 0.001$) and sparse categorical cross-entropy loss. While egg counting is a regression problem, we treated it as classification with $N = 43$ discrete classes (0 to 42 eggs). We did this because egg counts are integer-valued, and the softmax output lets the model express uncertainty across nearby count values instead of committing to a single number. The class prediction is:

$$\hat{y} = \arg\max_k \; P(y = k \mid x).$$

We also tried using the weighted expected value $\hat{y} = \sum_k k \cdot P(y = k \mid x)$ but found no consistent improvement over argmax at the cap level.

4.3.3 CNN Input Processing & Inference

At inference time, the system follows an aggregation pipeline. First, an image of a cap is provided. Next, the program normalizes the pixel intensities to match the pixel intensities used in training. The cap is then decomposed into tiles. The trained CNN then processes each 75 by 75 RGB tile and outputs a class prediction $\hat{y}$.

These individual tile predictions are stored in a temporary CSV file and then aggregated. The final egg count for a single cap is calculated as the sum of all tile-level predictions. This modular approach allows the system to identify specifically which regions of a cap contribute most to the total count, providing an extra layer of interpreting assay results. We developed a script to allow one to take these results and

overlay a density plot on the original cap to visualize the denser tiles.

The implementation is designed to integrate into laboratory workflows, as seen with the Command Line tool in Section 4.4. Once images are placed into the expected directory structure, the pre-processing, prediction, and aggregation steps take place.

4.4 Command Line Tool

To facilitate high-throughput analysis in a laboratory setting, we developed a Python-based command line interface (CLI). The tool automates the transition from raw imagery to structured data, a CSV file. The CLI consists of three primary stages:

1. **Pre-processing**: If the input images have not been previously partitioned, the tool invokes the image shredder module to segment the cap images into the tile format.

2. **Prediction**: The tool loads the user-specified or default model and performs a forward pass on all tiles in the provided directory.

3. **Aggregation**: A post-processing script is called to group the tile results by their parent image and calculate the final sums.

The tool supports a `--C` flag, which allows users to override the default model to use the specialized clustering model. If a custom model would like to be used instead, the tool supports a `--n` flag and the model path as input. It also supports the `-t` flag that allows a dataset containing tiles to be processed, too, The final output is a CSV file which provides a row-by-row egg count for every cap image processed. With the `--T` flag, a CSV file containing the counts assigned to each tile that the caps were decomposed into is also returned. The tool's `--o` flag allows the user to specify which output directory the CSV file(s) should be placed in.

5 Results

We evaluated model performance using mean square error (MSE), root mean square error (RMSE), and the coefficient of determination (R^2) at the cap level by summing tile-level predictions.

Table 2: Performance Metrics Comparison: 5-4 Dataset vs. CD Dataset

	5-4 Dataset			CD Dataset		
Training Set	**MSE**	**RMSE**	R^2	**MSE**	**RMSE**	R^2
4-30, 5-1, 5-2O, CC_A	490.99	22.16	0.2318	308.93	17.58	0.4448
4-30, 5-1, 5-2O	151.43	12.31	0.7631	237.62	15.41	0.5730
4-30, 5-1, 5-2S	138.34	11.76	0.7836	368.93	19.21	0.3370
4-30, 5-1, CC_A	118.26	10.87	0.8150	**204.29**	**14.29**	**0.6329**
4-30, 5-1	236.02	15.36	0.6307	337.89	18.38	0.3928
4-30, 5-2O	350.28	18.72	0.4519	337.16	18.36	0.3941
4-30, 5-2S	67.50	8.22	0.8944	609.18	24.68	-0.0948
4-30	118.28	10.88	0.8149	880.59	29.67	-0.5825
5-1, 5-2O	60.19	7.76	0.9058	342.26	18.50	0.3849
5-1, 5-2S, CC_A	154.19	12.42	0.7588	227.03	15.07	0.5920
5-1, 5-2S	**57.06**	**7.55**	**0.9107**	312.13	17.67	0.4391
5-1	297.52	17.25	0.5345	314.32	17.73	0.4351
5-2O	130.17	11.41	0.7963	925.73	30.43	-0.6636
5-2S	323.51	17.99	0.4938	931.48	30.52	-0.6740
CC_A, 4-30	228.58	15.12	0.6424	326.18	18.06	0.4138
CC_A	102104.25	319.54	-158.75	30444.10	174.48	-116.36
CC_J	23541.31	153.43	-35.83	59688.57	244.31	-9.7261
4-30, 5-1, 5-2S, CC_A, CC_J	290.82	17.05	0.5450	229.23	15.14	0.5881
4-30, 5-1, CC_A, CC_J	302.02	17.38	0.5275	237.08	15.40	0.5740
(Grayscale): 4-30, 5-1, 5-2O, CC_A	53251.18	230.76	-82.32	46660.42	216.01	-79.26

We define MSE, RMSE, and R^2 as follows:

- MSE $= \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$
- RMSE $= \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$
- $R^2 = 1 - \frac{\sum_{i=1}(y_i - \hat{y}_i)^2}{\sum_{i=1}(y_i - \frac{1}{n}\sum_{i=1}^{n} y_i)}$

Performance was assessed on two independent test sets: the Lithium **5-4 dataset**–which shares the same type of experimental assay as the data the models were trained on and contains cleaner images of caps–and the external **CD dataset** which provides a larger and messier benchmark.

5.1 Performance on the 5-4 Dataset

Across all the training configurations, models trained on the combinations of Lithium datasets substantially performed better than those trained on individual day or cluster-level datasets (Table 2).

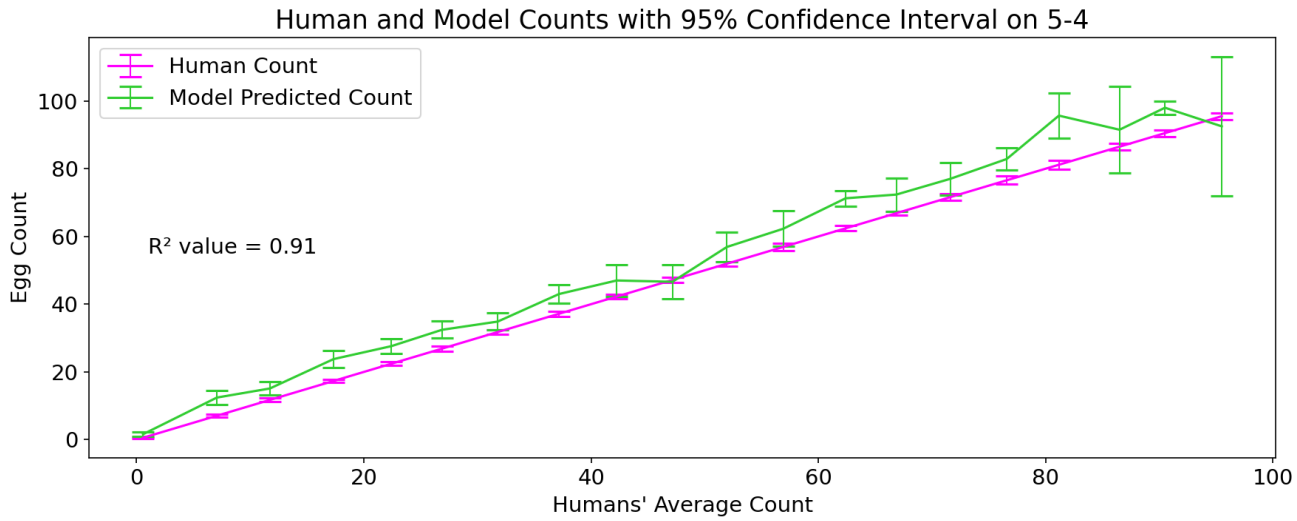


Figure 4: 95% Confidence Interval Average between Humans and the 5-1, 5-2S Model on 5-4 Dataset

The best-performing model on the 5-4 dataset was trained on **5-1 and 5-2S**, achieving an MSE of 57.06, meaning that the model was off by around 7.55 eggs on the caps on average, and an R^2 value of 0.9107. The R^2 values indicates that over 91% of the variance in true egg counts was explained by the model.

Several other multi-day training configurations also yielded in low RMSEs. For example, training on **4-30, 5-2S** achieved a RMSE of 8.22, while **5-1, 5-2O** achieved a RMSE of 7.76. On the other hand, models trained on single-day datasets consistently performed badly. Due to this different, we believe that inter-day variation in egg distribution on tiles and imaging conditions led to the model to be able to generalize on the testing set.

Models trained with cluster-heavy datasets (**CC_A**) performed more poorly than those without cluster sets. This is due to the fact that the 5-4 dataset is a dataset with very few clusters. The models with cluster datasets tended to misclassify yeast smears or light reflections on a cap as clusters, leading to a greater RMSE value as eggs were over-counted. This indicates that cluster-only data is insufficient for learning the full distribution of tile-level egg counts and severely biases predictions when used without lower count egg tiles.

We also tested a grayscale version of the model trained on the 4-30, 5-1, 5-2O, and CC_A datasets to see whether color information was necessary for egg detection. The grayscale model performed extremely poorly, achieving an MSE of 53,251.18 on the 5-4 dataset and 46,660.42 on the CD dataset, with

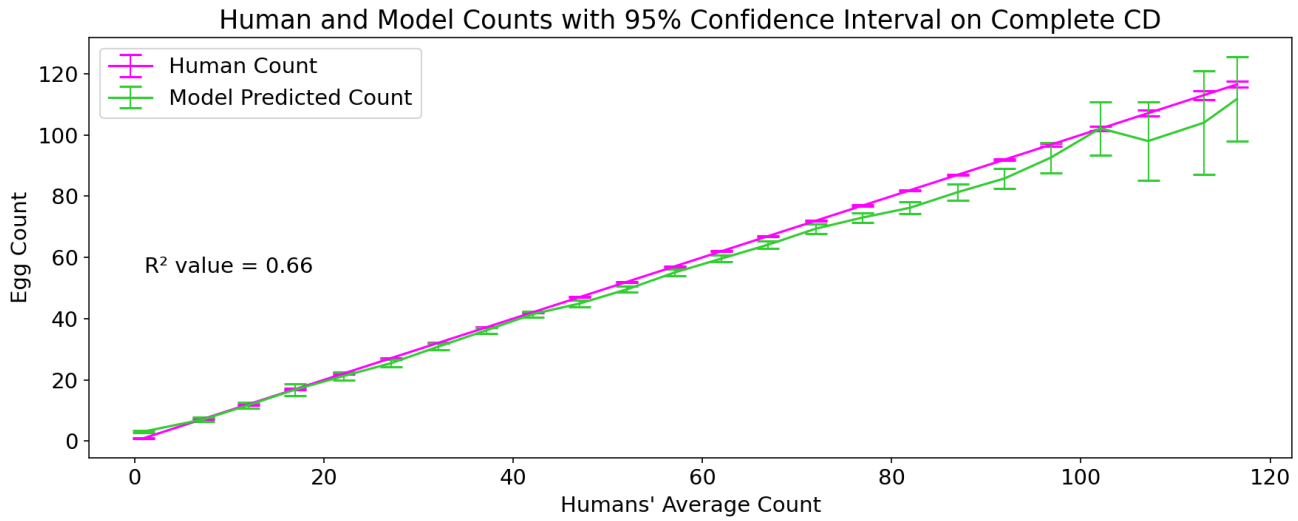


Figure 5: 95% Confidence Interval Average between Humans and the 4-30, 5-1, CC_A Model on the Complete CD Dataset without the two human miscounts, which changed the R^2 value to 0.66.

negative R^2 values of -82.32 and -79.26 respectively. These results are far worse than random and indicate that converting tiles to grayscale before training causes the model to lose the contrast information it relies on to distinguish the white eggs from the dark background. This confirms that RGB color information is essential for the tiling approach to work, and that grayscale pre-processing should not be used.

Additionally, the 5-4 dataset was specifically curated based on the counts of two humans. In doing this, we are able to compare the model's predictions relative to the people. Based on this result, we noticed that the model tends to over-count, especially at higher cap counts. We reason this to be due to two reasons. First, the human data the model was trained on is an inherently over-counted set since Alexander tends to over-count. Therefore, the model developed a bias towards over-counting eggs. Second, clusters occur in high-egg-count caps, so the model is over-counting more at higher cap counts. However, overall, the model still performs close to the human's counts. We conducted a paired t-test on the model's predicted counts and the average human's counts since both the model and human counts are for the same tiles and full caps. The results showed statistically significant results with a p-value less than 0.01 in only 9 bins as seen in Table 3.

Table 3: Statistical Significance Analysis of 5-4 Dataset with 5-1, 5-2S Model

Actual Cap Count Bin	Model Mean	Human Mean	t-statistic	p-value	Significance
$[0.0, 5.0)$	1.62	0.29	4.347	0.0000	***
$[5.0, 10.0)$	12.43	3.53	4.988	0.0000	***
$[10.0, 15.0)$	15.12	5.87	3.620	0.0013	**
$[15.0, 20.0)$	23.79	8.67	5.236	0.0000	***
$[20.0, 25.0)$	27.61	11.21	4.259	0.0002	***
$[25.0, 30.0)$	32.44	13.44	4.568	0.0004	***
$[30.0, 35.0)$	34.87	15.89	2.285	0.0323	*
$[35.0, 40.0)$	43.00	18.59	4.210	0.0007	***
$[40.0, 45.0)$	47.00	21.12	2.002	0.0637	
$[45.0, 50.0)$	46.62	23.58	-0.211	0.8367	
$[50.0, 55.0)$	56.89	25.97	2.402	0.0280	*
$[55.0, 60.0)$	62.27	28.45	1.971	0.0770	
$[60.0, 65.0)$	71.27	31.20	6.910	0.0000	***
$[65.0, 70.0)$	72.36	33.43	2.244	0.0429	*
$[70.0, 75.0)$	77.00	35.80	2.105	0.0646	
$[75.0, 80.0)$	82.86	38.29	3.187	0.0189	*
$[80.0, 85.0)$	95.67	40.58	4.768	0.0050	**
$[85.0, 90.0)$	91.50	43.25	0.714	0.6051	
$[90.0, 95.0)$	98.00	45.25	15.000	0.0424	*
$[95.0, 100.0)$	92.50	47.75	-0.273	0.8305	

Significance levels: *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$

5.2 Performance on the CD Dataset

When evaluated on the CD dataset, overall performance declined relative to the 5-4 dataset, as expected due to differences in experimental conditions and image quality. Despite this, some models had moderate predictive power. The best-performing model was trained on **4-30, 5-1, and CC_A**. That model had the lowest MSE of 204.29, RMSE of 14.29, and highest R^2 of 0.6329.

The training sets that performed best on the 5-4 dataset did not translate into being the best models on the CD dataset. For example, the **5-2, 5-2S** model achieved a R^2 of 0.4391 on the CD dataset despite achieving an R^2 of 0.9107 on the 5-4 dataset. Conversely, the inclusion of cluster data alongside the Lithium datasets improved robustness to the distributional shift due to the wider range of egg densities in the CD dataset.

Moreover, the CD dataset contains many caps with yeast "smears" due to human error in taking images and properly taking care of caps. The 4-30, 5-1, and CC_A model tended to predict those smears

Table 4: Statistical Significance Analysis of CD Dataset with 4-30, 5-1, CC_A Model

Actual Cap Count Bin	Bot Mean	Human Mean	t-statistic	p-value	Significance
$[0.0, 5.0)$	3.15	0.49	10.693	0.0000	***
$[5.0, 10.0)$	7.14	3.65	-0.506	0.6139	
$[10.0, 15.0)$	11.75	6.00	-0.528	0.5978	
$[15.0, 20.0)$	16.87	8.50	-0.124	0.9011	
$[20.0, 25.0)$	21.30	11.07	-1.291	0.1981	
$[25.0, 30.0)$	25.47	13.55	-2.831	0.0050	**
$[30.0, 35.0)$	30.87	16.05	-2.281	0.0231	*
$[35.0, 40.0)$	36.17	18.57	-1.911	0.0567	
$[40.0, 45.0)$	41.48	21.04	-1.171	0.2422	
$[45.0, 50.0)$	44.94	23.51	-4.159	0.0000	***
$[50.0, 55.0)$	49.56	26.01	-5.108	0.0000	***
$[55.0, 60.0)$	55.09	28.53	-3.950	0.0001	***
$[60.0, 65.0)$	59.60	31.00	-4.525	0.0000	***
$[65.0, 70.0)$	64.14	33.47	-4.578	0.0000	***
$[70.0, 75.0)$	69.37	35.99	-3.384	0.0008	***
$[75.0, 80.0)$	73.02	38.47	-5.017	0.0000	***
$[80.0, 85.0)$	76.20	40.96	-5.864	0.0000	***
$[85.0, 90.0)$	81.30	43.49	-4.218	0.0000	***
$[90.0, 95.0)$	85.80	45.97	-3.824	0.0002	***
$[95.0, 100.0)$	92.57	48.40	-1.663	0.1033	
$[100.0, 105.0)$	102.18	51.03	0.026	0.9800	
$[105.0, 110.0)$	98.00	53.56	-1.373	0.2070	
$[110.0, 115.0)$	104.00	56.50	-1.115	0.3462	
$[115.0, 120.0)$	111.75	58.25	-0.658	0.5573	

Significance levels: *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$

as clusters sometimes, leading to over-counting. However, at the same time, there were many human error in counting the caps in the CD dataset. In our analysis in Table 4 and Figure 5, we removed the two human errors during transcription from physical bookkeeping to Excel spreadsheets, where caps were miscounted as having 296 and 438 eggs.

Since the CD dataset was based on the work done in Phillips et al., 2018, we recreated the plot of the difference between the C and D-type *Drosophila melanogaster* based on the fecundity counts the best-performing model outputted. Based on Figure 6, we can see that our model's counts are similar to the actual human counts, preserving the direction and magnitude of the C vs. D-type fecundity difference found in the original study. This is notable because the model was never exposed to CD dataset images during training, yet still recovers the same population-level trend that the original human counters found.

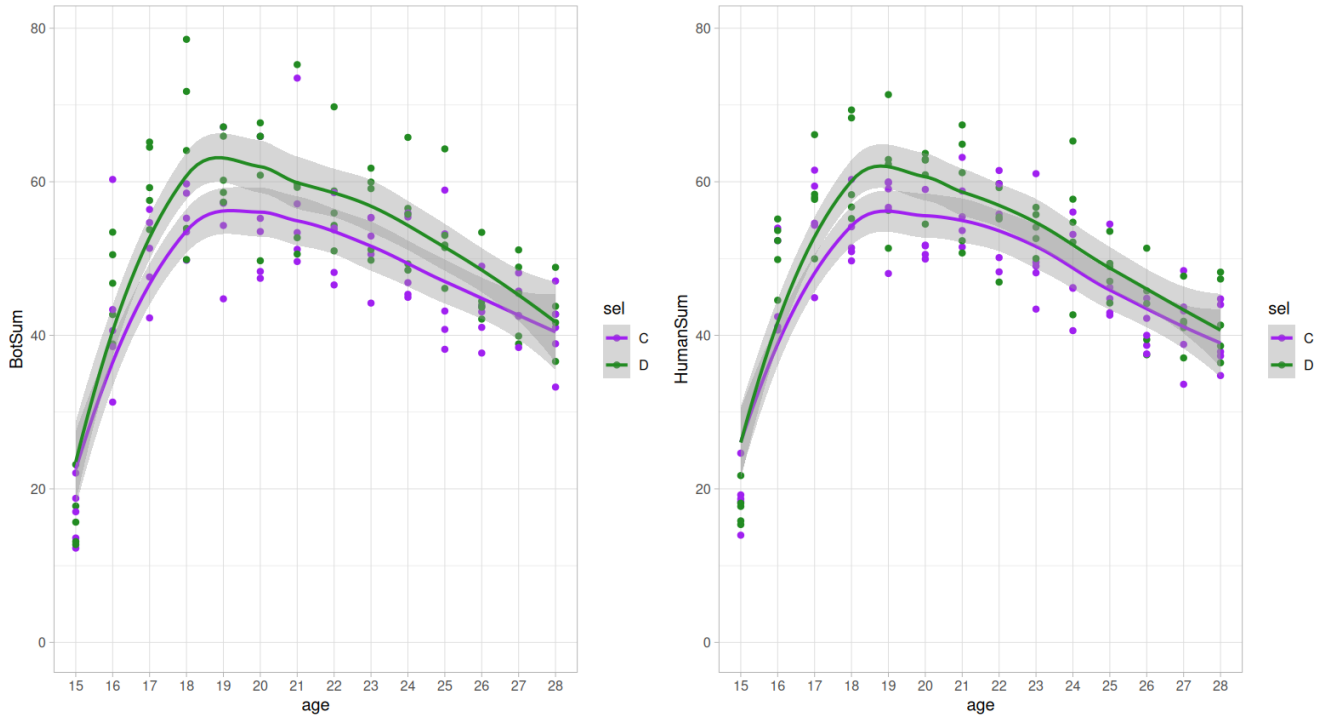


Figure 6: CD results based on the 4-30, 5-1, CC_A model's count (BotSum) vs. the actual counts (HumanSum) from Phillips et al., 2018.

This suggests that even at an RMSE of 14.29 eggs per cap, the model's errors are not systematic with respect to population type and do not obscure the biologically meaningful difference between C and D-type populations. Therefore, CNN-based automation is a viable approach for large-scale fecundity experiments where the primary goal is detecting differences between populations rather than obtaining exact counts for individual caps.

We also conducted a paired t-test on the CD dataset between the model's predicted counts and the human counts, the results of which are shown in Table 4. Compared to the 5-4 dataset, the CD model shows far fewer statistically significant bins. On the 5-4 dataset, the model was significantly different from the human counts in the majority of bins, reflecting the over-counting bias from Alexander's training data. On the CD dataset, the model was not significantly different from the human counts in the low-to-mid count range, with significant differences only appearing consistently from the $[45.0, 50.0)$ bin onward. This is somewhat counterintuitive given that the CD dataset had a lower R^2 of 0.6329 compared to 0.9107 on the 5-4 dataset. The reason for this is that the CD dataset has much higher cap-level variance than the 5-4 dataset, which widens the confidence intervals enough that the model's predictions are not significantly different from the human counts even when individual cap predictions are noisier. In other

words, the CD dataset is harder to count in general, so the model's errors are harder to distinguish from human disagreement. The bins above $[95.0, 100.0)$ also show no significant difference, but this is likely due to the small number of caps in those bins rather than the model performing well at high counts.

5.3 Model and Human Count Agreement with 95% Confidence Intervals

Figures 4 and 5 plot the model's predicted counts and the human average counts against the humans' average count, with 95% confidence intervals, for the best-performing models on each test set.

On the 5-4 dataset ($R^2 = 0.91$), the human count line represents the average of two independent counters (Julie and Angela), providing a more stable ground truth than a single annotator. The model tracks this average closely at low egg counts but diverges progressively at higher counts, with the model's green curve pulling above the human average's magenta curve beginning around 20 eggs. This upward divergence grows with cap count and is consistent with the over-counting bias identified in Table 3, likely stemming from the model having been trained predominantly on Alexander's counts, which carry a systematic over-counting tendency not present in Julie and Angela's annotations. Notably, the confidence intervals of the model and human average overlap substantially throughout the low-to-mid count range, suggesting the model's predictions fall within the range of human variability for the majority of caps. At higher counts, the model's confidence intervals widen considerably, reflecting increased prediction variance on caps with dense egg regions where cluster tiles are more prevalent.

On the CD dataset ($R^2 = 0.66$), the two curves remain much closer together across the full count range compared to the 5-4 dataset, with the model and human confidence intervals overlapping through approximately 100 eggs. This tighter agreement at the mean level is somewhat counterintuitive given the lower R^2, but can be explained by the increased cap-level variance in the CD dataset: the wider confidence intervals reflect greater noise in individual predictions rather than a systematic directional bias. Beyond 100 eggs, the model's confidence intervals expand sharply and the curves begin to diverge, likely due to the relative scarcity of high-count caps in the training data and the presence of yeast smears that the model occasionally misclassified as clusters.

5.4 Summary of Findings

Overall, these results demonstrate that the CNN can accurately estimate female fecundity for *Drosophila melanogaster* using the tiling technique when trained on a diverse, representative tile dataset. Performance is strongest the training data spans across multiple collection days and includes both non-cluster and clustered data when the day's images contain dense regions of eggs, as well. The model retains meaningful predictive power, which means that the tiling-based CNN approach generalizes beyond a single assay and holds promise for scalable, automated fecundity estimation in large experimental pipelines.

6 Discussion

Our results demonstrate that CNNs can successfully automate fecundity assays in *Drosophila melanogaster* populations with the tiling approach. The best models achieved RMSEs of 7.55 eggs on the 5-4 dataset and 14.29 eggs on the CD Dataset. These results show improvement over the time efficiency required for manual counting while maintaining reasonable accuracy. Therefore, CNN-based automation fixes the time-related issue in large-scale fecundity assays.

The tiling approach we used was able to overcome the many classes problem present when training used complete caps. In decomposing caps into 75 by 75 pixel tiles, we reduced the classification task from hundreds of potential classes (up to 200 eggs per cap with highly variable spatial distributions) to 43 classes (up to 42 eggs per tile). This reduction allowed the CNN to learn robust features for egg detection without requiring prohibitively large training dataset. The success of tiling shows that decomposition strategies can improve model performance on complex counting tasks and small object detection.

Additionally, we trained models based on a specific person's counts in a dataset. Developing a model trained on a single person reduces the biases and differences in counting styles of different people. Models trained on multiple different people performed worse than those trained on a single person's data. For instance, the 4-30, 5-1, CC_A, CC_J dataset had an RMSE of 15.40 eggs while the 4-30, 5-1, CC_A dataset had an RMSE of 14.29 eggs on the CD dataset as seen in Table 2. This shows that consistency between the source of training data improved the performance of models. Similarly, training the model on multiple days improved the RMSE of models. Models that were trained on combinations of datasets

from different collect days consistently outperformed those trained on single-day datasets alone. For example, training only on the 5-1 dataset yielded an RMSE of 17.25 on the 5-4 test set, while combining 5-1 with 5-2S achieved an RMSE of 7.55. This suggests that inter-day variation in imaging conditions, egg distribution, and other factors provides regularization that helps model generalize to new data.

The inclusion of cluster data had different effects on performance depending on which test dataset was used. The 5-4 dataset contained relatively few clusters, adding cluster-specific training data (CC_A) generally degraded performance and instead introduced an over-counting bias where whitespace became classified as clusters. However, on the CD dataset, which contained more high-density egg regions and visual noise from yeast smears, models trained with cluster data performed better. The 4-30, 5-1, CC_A model achieved the best performance on the CD dataset with an RMSE of 14.29. This suggests that cluster data is useful when the target domain includes dense regions on caps but is harmful when applied to cleaner, sparse caps.

One challenge we encountered was the model's tendency to over-count, especially at high cap counts. Based on our statistical analysis of the 5-4 dataset counters' results and the model's predictions, we found significant over-counting in the higher count bins. We believe this is due our training data which comes from Alexander, who has a systematic over-counting bias, which the model replicated.

The performance gap between the 5-4 and CD datasets shows the importance of training data similarity to deployment conditions. The 5-4 dataset was collected under the same experimental protocol as the training data, leading to better results (R^2 = 0.91) than the CD dataset (R^2 = 0.63). Moreover, the CD dataset had more imaging artifacts, yeast smears, variable lighting conditions, and the mislabeled human counts in the test set–all of which simulated how real data can look like. This led to degraded model performance but did not eliminate its predictive power. The model was still able to reproduce the C vs. D-type fecundity difference observed in the original study as seen in Figure 6. This suggests that even with reduced accuracy, the CNN can still capture biologically meaningful population-level trends.

Inter-Rater Agreement and Ground Truth Quality

The Cohen's Kappa of $\kappa = 0.7292$ for the 5-4 dataset is worth discussing because it sets a ceiling on how accurate any model trained on these labels can be. A κ of 0.73 means that even trained human counters

disagree on a meaningful fraction of tiles, especially cluster tiles where egg boundaries are hard to see. This disagreement comes from a few main sources:

- Eggs that are partially outside the tile boundary.
- Partially hatched or desiccated eggs that do not look like normal white ellipses.
- High-density cluster tiles where individual eggs are hard to tell apart.

Because of this, an RMSE of 7.55 eggs on the 5-4 dataset does not mean the model is far off from the truth. Part of the error comes from the fact that the training labels came from Alexander, who over-counts, while the test set was counted by Julie and Angela, who do not share that bias.

Mitigating the Over-Counting Bias

The over-counting bias in the model comes from Alexander's counting style being in the training data. There are a few ways this could be fixed in future work. First, labels could be collected from multiple counters and averaged before training, which would remove the bias from any one person. Second, a simple linear calibration could be fit between model predictions and averaged human counts on a held-out set, correcting the offset without retraining. Third, a secondary calibration layer could be added to the network to learn a correction function alongside the main counting objective. Of these, the linear calibration is the most practical since it only requires a small labeled calibration set and no changes to the training pipeline.

Compared to previous automation approaches like *QuantiFly*, our CNN-based method offers several advantages. While *QuantiFly* relied on density estimation with specific imaging requirements, our approach learns features directly from data and is thus robust across different imaging conditions (Waithe et al., 2015). However, recent work using YOLOv8 in MLDAAP by Gabidulin and Rudman, 2023 achieved 91% accuracy through real-time object detection, which is better for certain assays. The choice between these approaches depends on the specific assay.

7 Conclusion

There are many practical implications of this work but some limitations do exist.

To reiterate, for an experiment comparing 20 populations with 500 flies per population over 20 days (200,000 images), manual counting at 3 minutes per image would require over a year (417 days). Our automated pipeline reduces this to 5 days of computational time on our computer, a reduction of roughly 99% in time spent counting. This frees up lab members to focus on maintaining populations and designing experiments rather than counting eggs. The command line tool also makes this accessible to labs without machine learning expertise, since once images are placed in the expected directory, the tool handles the rest and outputs a CSV of egg counts.

Beyond saving time, automating counting also reduces inter-rater variability. As seen with the 5-4 dataset, two trained human counters had a Cohen's Kappa of 0.7292, meaning there is real disagreement even between careful annotators. A model counts consistently every time, which is especially useful in large experiments where counting is split across multiple research assistants over many days, since differences in counting style between people can introduce systematic error into the data.

However, we have several limitations. First, our models were trained and tested only on data from our laboratory with a specific imaging setup. Whether the model works in other labs with different lighting, cameras, or cap colors is untested, and those labs would likely need to collect their own training data. Second, the tile-based approach discards spatial information about where eggs are located on the cap, which may matter for assays interested in egg distribution rather than just total counts. Third, the model treats all eggs equally, but some assays may need to distinguish between egg types or developmental stages, which would require different training data and changes to the architecture. Finally, the model tends to over-count at high cap counts, which is a limitation for experiments with high-fecundity populations where caps regularly exceed 60 eggs.

Future work could go in a few directions. Collecting training data from multiple counters and averaging their labels before training would reduce the annotator bias we saw from Alexander's counts. Applying a simple post-processing calibration to the model's outputs could also fix the over-counting bias without retraining. Incorporating spatial position of tiles or passing the full cap image to the model might improve performance on clustered regions. Adding an uncertainty estimate to the model's output would also let it flag low-confidence caps for human review, giving labs a way to combine automated counting with human oversight on the harder cases.

In conclusion, our CNN-based approach with tiling was able to successfully automate fecundity assay counting, achieving biologically meaningful accuracy while reducing manual labor by orders of magnitude. The model works best when trained on data from multiple collection days by a consistent counter, and when cluster data is included if the target assay has dense egg regions. While limitations remain, the results on both the Lithium and CD datasets show that the model recovers the same biological conclusions as human counters in a fraction of the time, and has immediate practical applications for any lab running large-scale fecundity experiments.

8 Acknowledgments

I want to thank Zachary Greenspan for his guidance and mentorship throughout my time at Rose Lab. I want to thank Dr. Michael Rose for showing me what it means to be a scientist and inspiring me. Lastly, I want to thank Dr. Larry Mueller for investing in my growth as a scientist.

References

Buitinck, L., Louppe, G., Blondel, M., Pedregosa, F., Mueller, A., Grisel, O., Niculae, V., Prettenhofer, P., Gramfort, A., Grobler, J., Layton, R., VanderPlas, J., Joly, A., Holt, B., & Varoquaux, G. (2013). API design for machine learning software: Experiences from the scikit-learn project. *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, 108–122.

Djawdan, M., Sugiyama, T. T., Schlaeger, L. K., Bradley, T. J., & Rose, M. R. (1996). Metabolic aspects of the trade-off between fecundity and longevity in drosophila melanogaster. *Physiological Zoology*, *69*(5), 1176–1195. Retrieved January 9, 2026, from http://www.jstor.org/stable/30164252

Gabidulin, A. R., & Rudman, S. M. (2023). Mldaapp: Machine learning data acquisition for assessing population phenotypes. *G3: Genes — Genomes — Genetics*, *15*. https://api.semanticscholar.org/CorpusID:263153835

Landis, J. R., & Koch, G. G. (1977). The measurement of observer agreement for categorical data. *Biometrics*, *33*(1), 159–174. Retrieved May 12, 2026, from http://www.jstor.org/stable/2529310

Li, Y., Zhang, X., & Chen, D. (2018). Csrnet: Dilated convolutional neural networks for understanding the highly congested scenes. *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 1091–1100. https://api.semanticscholar.org/CorpusID:3645757

Lürig, M. D., Donoughe, S., Svensson, E. I., Porto, A., & Tsuboi, M. (2021). Computer vision, machine learning, and the promise of phenomics in ecology and evolutionary biology. *Frontiers in Ecology and Evolution*, *Volume 9 - 2021*. https://doi.org/10.3389/fevo.2021.642774

Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Jia, Y., Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, … Xiaoqiang Zheng. (2015). TensorFlow: Large-scale machine learning on heterogeneous systems [Software available from tensorflow.org]. https://www.tensorflow.org/

Mueller, L., & Rose, M. (2007). An evolutionary heterogeneity model of late-life fecundity in drosophila. *Biogerontology*, *8*, 147–61. https://doi.org/10.1007/s10522-006-9042-x

Phillips, M., Rutledge, G., Kezos, J., Greenspan, Z., Talbott, A., Matty, S., Arain, H., Rose, M., & Shahrestani, P. (2018). Effects of evolutionary history on genome wide and phenotypic convergence in drosophila populations. *BMC Genomics*, *19*. https://doi.org/10.1186/s12864-018-5118-7

Rauser, C. L., Mueller, L. D., & Rose, M. R. (2003). Aging, fertility, and immortality [Proceedings of the 6th International Symposium on the Neurobiology and Neuroendocrinology of Aging]. *Experimental Gerontology*, *38*(1), 27–33. https://doi.org/https://doi.org/10.1016/S0531-5565(02)00148-1

Ronneberger, O., Fischer, P., & Brox, T. (2015). U-net: Convolutional networks for biomedical image segmentation. https://arxiv.org/abs/1505.04597

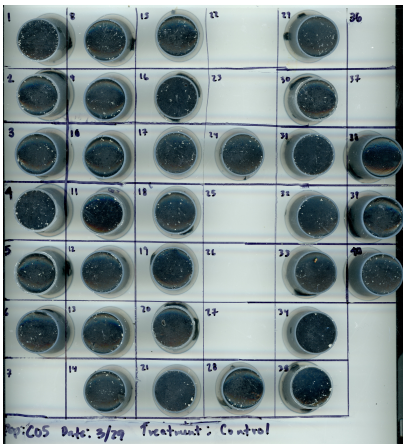
Rutledge, G. A., Cabral, L. G., Kuey, B. J., Lee, J. D., Mueller, L. D., & Rose, M. R. (2020). Hamiltonian patterns of age-dependent adaptation to novel environments. *PLOS ONE*, *15*(10), 1–12. https://doi.org/10.1371/journal.pone.0240132

Wadgymar, S., Sheth, S., Josephs, E., DeMarche, M., & Anderson, J. (2024). Defining fitness in evolutionary ecology. *International Journal of Plant Sciences*, *185*. https://doi.org/10.1086/729360

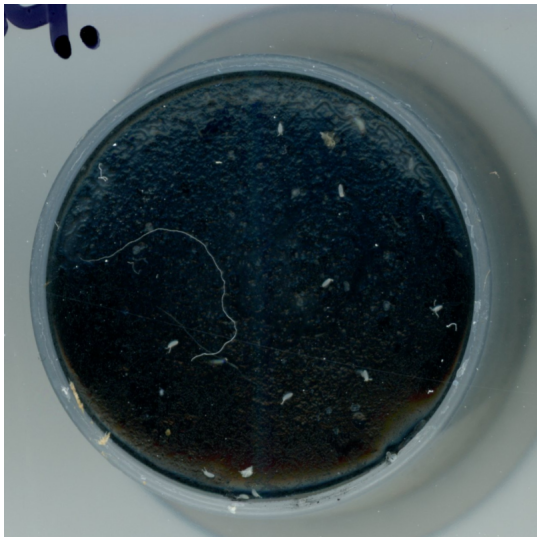
Waithe, D., Rennert, P., Brostow, G. J., & Piper, M. D. W. (2015). Quantifly: Robust trainable software for automated drosophila egg counting. *PLoS ONE*, *10*. https://api.semanticscholar.org/CorpusID:17866756

Zhang, Y., Zhou, D., Chen, S., Gao, S., & Ma, Y. (2016). Single-image crowd counting via multi-column convolutional neural network. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 589–597. https://doi.org/10.1109/CVPR.2016.70

A Appendix



(a) Grid Image Example



(b) Cap Image Example

Figure 7: Example grid and cap images. During inference, the grid image is decomposed into cap images with the command line tool.



(a) Whitespace Example



(b) One Egg Example



(c) Two Eggs Example



(d) Six Eggs Example



(e) Eight Eggs Example



(f) Cluster Tile Example

Figure 8: Example tile images, where the cluster tile contains 42 eggs.